

Web Exploitation 101

The Hacker Mindset

What is Web Exploitation?

"Finding the gap between what the developer intended and what the application actually allows."

Core Philosophy:

Everything is Input: Parameters, headers, cookies, files, API calls

Trust Nothing: Assume all data can and will be manipulated

Think in Chains: One vulnerability often leads to another

The Hacker's Golden Rule:

"If you can make it do something it wasn't designed to do, you've found an exploit."

Phase 1: Foundation & Mindset (Pre-Exploitation) Target: Understand what you're attacking.

Step 1: Recon & Mapping

Tool: Browser Developer Tools (F12), nmap, whois, gobuster, dnsmap, dnsrecon, whatweb etc.

Map all pages and inputs (forms, search bars, URL parameters).

Identify the tech stack (PHP? Apache? A framework?).

Look for hidden files (robots.txt, .git/).

Step 2: Setup Your Lab

Tool: DVWA (Damn Vulnerable Web Application).

Why: A safe, legal, and configurable environment to practice. Install it locally using Docker or XAMPP.

Phase 2: The Web Exploitation Toolkit (Complete List) A. INJECTION ATTACKS - Breaking the Logic

1. SQL Injection (SQLi) - The Database Killer What: Injecting malicious SQL through user input Impact: Data theft, authentication bypass, full database compromise Real-world Example: Heartland Payment Systems (2008) - 130 million cards stolen

example payload ' OR '1'='1' -- UNION SELECT username, password FROM users--

Types: Classic/Union-based: Direct database interaction

Blind SQLi: No direct output - infer from behavior

Time-based: Use SLEEP() commands to detect

Second-order: Stored for later execution

2. Cross-Site Scripting (XSS) - Client-Side Takeover What: Injecting scripts that execute in victims' browsers Impact: Session theft, keylogging, defacement, malware distribution

The XSS Trinity:

Reflected XSS: Malicious script in URL/input → Immediate execution

Stored XSS: Script saved in database → Affects all visitors

DOM-based XSS: Client-side JavaScript mishandling → No server contact needed

example advanced payload

3. Server-Side Template Injection (SSTI) - Remote Code Execution What: Injecting template directives into user input Impact: Remote code execution, server takeover Targets: Jinja2 (Python), Twig (PHP), Freemarker (Java)

Detection: `{{7*7}}` → If output shows "49", vulnerable Exploitation:

`{{config.class.init.globals['os'].popen('whoami').read()}}`

4. Command Injection - Shell Access What: Injecting OS commands through application input Impact: Full server compromise

example payload ; cat /etc/passwd | ls -la whoami \$(id)

5. XXE (XML External Entity) - File & Network Access What: Abusing XML parsers to read files or make network calls Impact: File disclosure, SSRF, denial of service

example payload]> &xxe;

B. BROKEN ACCESS CONTROL - The Permission Problem 6. IDOR (Insecure Direct Object Reference) - The Easy Win What: Accessing objects by manipulating identifiers Impact: Data breach, unauthorized actions Pattern: /user/profile?id=123 → Change to id=124 Modern Variant: Mass Assignment - Setting unauthorized fields via API

7. CSRF (Cross-Site Request Forgery) - The Forged Request What: Tricking users into submitting malicious requests Impact: Account takeover, financial fraud

Defense: Anti-CSRF tokens, SameSite cookies

Attack Flow: User logs into bank.com User visits malicious site Site auto-submits:

8. SSRF (Server-Side Request Forgery) - The Pivot Attack What: Making the server request internal resources Impact: Internal network enumeration, cloud metadata theft

Targets:

Internal services: <http://localhost:8080/admin>

Cloud metadata: <http://169.254.169.254/latest/meta-data/>

File reading: <file:///etc/passwd>

Bypass Techniques: <http://127.0.0.1> → <http://0177.0.0.1> (octal) <http://localhost> → <http://localtest.me> (DNS rebinding)

C. INFORMATION DISCLOSURE - Finding Hidden Secrets 9. Directory Traversal - The File Explorer What: Accessing files outside web root Impact: Source code disclosure, configuration leaks

example payload ../../../../etc/passwd %2e%2e%2f%2e%2e%2fetc/passwd (URL encoded)

10. Sensitive Data Exposure Sources: .git/ directory - Full source code history .env files - API keys, database credentials Backup files: database.sql.bak, config.php.old Comments in source code Error messages with stack traces

D. AUTHENTICATION & SESSION ATTACKS 11. Session Hijacking & Fixation Methods: XSS + Cookie Theft: `document.cookie` Session Prediction: Sequential session IDs Session Fixation: Forcing user to use attacker's session ID

12. Brute Force & Credential Stuffing Tools: Hydra, Burp Intruder, custom scripts

Defenses: Rate limiting, CAPTCHA, MFA

13. OAuth & JWT Vulnerabilities JWT: None algorithm attack, weak secrets, kid header injection

OAuth: Open redirects, scope manipulation, CSRF in auth flow